



Ledger

Crash-safe embedded state storage without third-party runtime dependencies

Zero Dependency Hackathon

Track D — Data & Storage

Venugopal1807

0

third-party imports

369

tests passing

12,734

hostile recovery
cases

15,453 B

standalone artifact

The problem is interruption

Small local applications persist state with whatever is closest to hand.

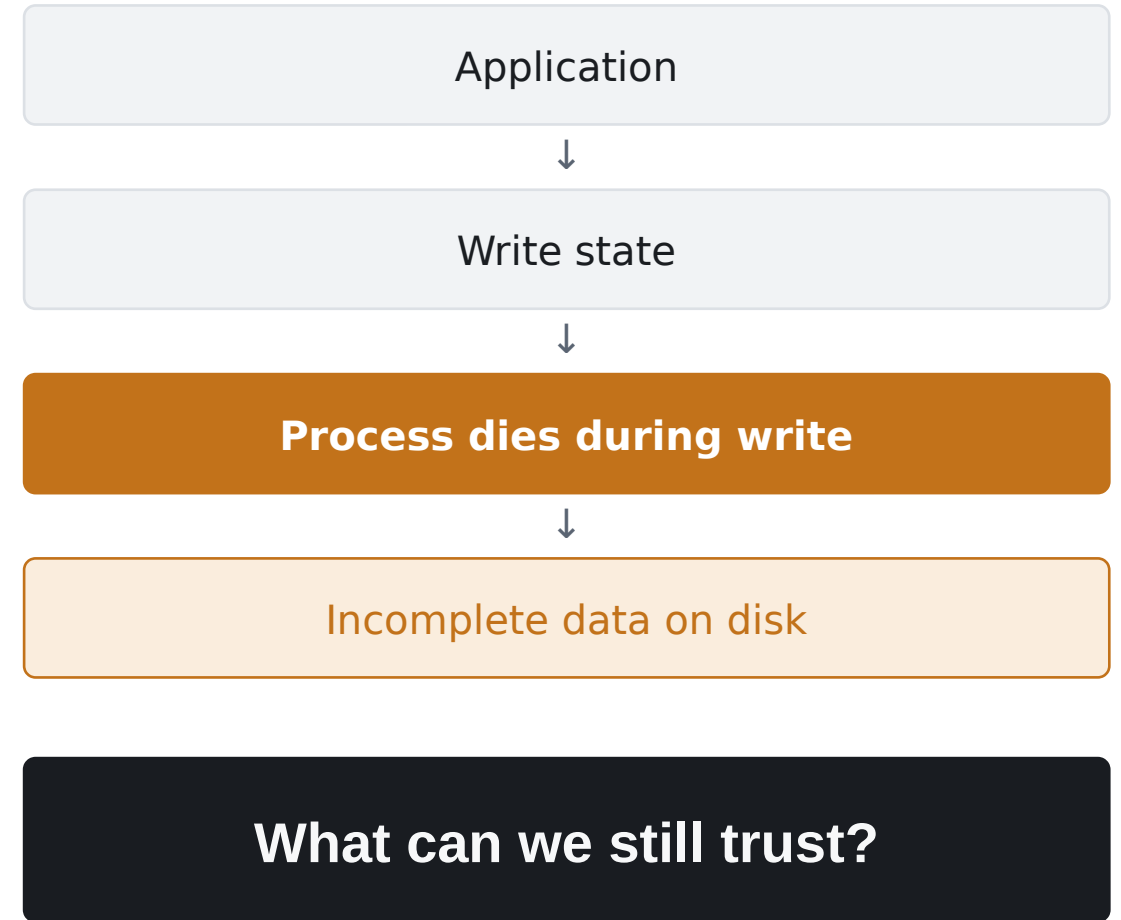
How local state is usually stored

- JSON files
- shelve
- diskcache-style persistence
- simple file-based state

The happy path is easy:

```
json.dump(state, open("state.json", "w"))
```

`open(..., "w")` truncates the file first, then rewrites it. Between those two moments the old state is already gone and the new state is not there yet.

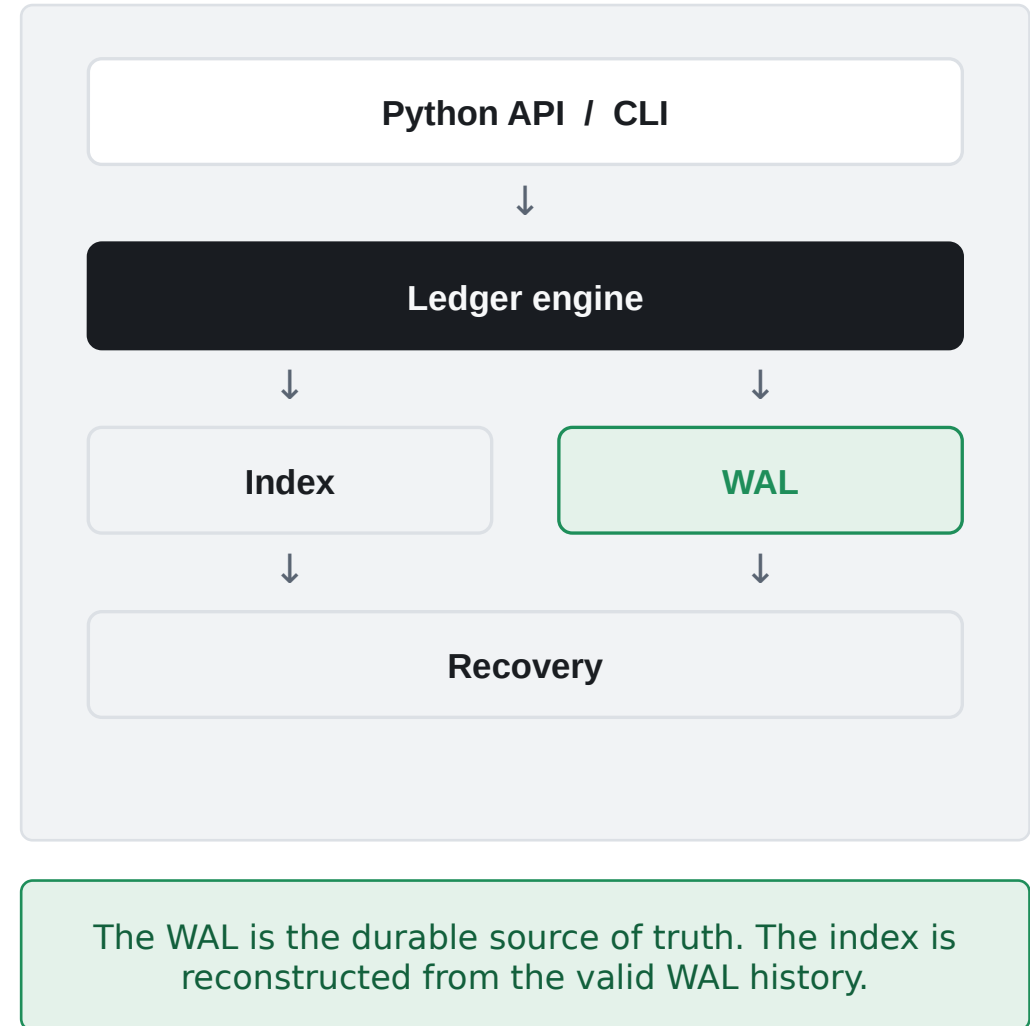


Most stores answer this by accident. Ledger answers it by design.

Append, checksum, never rewrite

Seven mechanisms, each protecting one property.

- **Append-only WAL**
No committed byte is ever rewritten, so a crash can only damage the tail.
- **Fixed binary record framing**
32-byte header, length-prefixed key and value.
- **Header CRC-32**
Verified before any length field is trusted.
- **Payload CRC-32**
Covers key and value bytes independently.
- **Sequence continuity**
Strict +1 per record; gaps and regressions are detected.
- **Conservative recovery**
Stops at the first invalid record and never scans forward.
- **In-memory index**
Current state, rebuilt from the valid WAL history on open.



Crash recovery

A writer committed four records, wrote 20 bytes of a fifth, and killed itself.



4 committed records — provably valid prefix

partial 5th — 20 of 49 bytes

- 4 valid records
- partial 5th record
- SIGKILL
- reopen
- **valid prefix preserved**
- torn tail rejected
- **automatic repair**

ACTUAL DEMO EVIDENCE

```
python3 demo/crash_recovery.py
```

```
committed before the kill : 4 records
partial record written      : 20 of 49 bytes
child exit status           : -9 (SIGKILL)
```

```
$ python3 -m ledger inspect crash.ledger
```

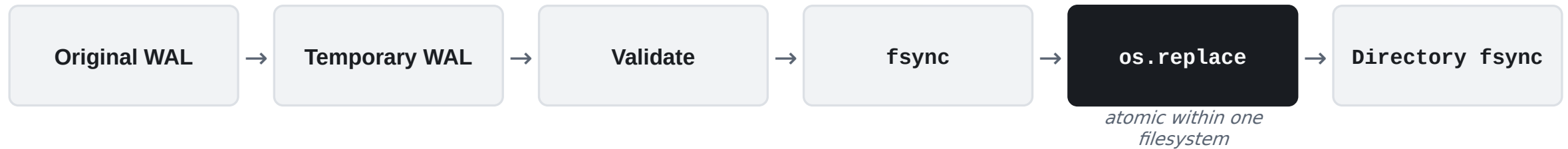
```
file size:      380 B
valid records:   4
live keys:      4
valid end offset: 360
tail state:      TORN
tail reason:     short_header
discarded bytes: 20
repair required: yes
```

*Ledger never guesses what an incomplete record meant.
It keeps the valid prefix and rejects the untrusted tail.*

After a write-mode open: 5 records · 409 B · tail CLEAN

Compaction that cannot lose the log

The original WAL is never rewritten in place and never truncated.



Measured on the demo store

records	38	→	6
dead bytes	1778 (84.1%)	→	0 (0.0%)
file size	2.1 KiB	→	368 B
live keys	6	→	6

live keys unchanged: state is preserved, only obsolete versions are dropped

Killed mid-compaction

before replace **original WAL survives**
26 valid records · 4 live keys · CLEAN

after replace **compacted WAL survives**
4 valid records · 4 live keys · CLEAN

Nine named interruption points around the compaction state machine.

Across the tested interruption points, reopening produced either the original valid WAL or the fully replaced compacted WAL. No mixed authoritative WAL was observed.

Zero dependencies, mechanically enforced

The audit reads source, not the environment: pip freeze describes a machine.

third-party imports **0**

unresolved imports **0**

python3 tools/depcheck.py → exit 0

ledger.py imports ten standard-library modules and nothing else

`__future__`

`argparse`

`dataclasses`

`fcntl`

`json`

`os`

`pathlib`

`struct`

`sys`

`zlib`

The audit is implemented twice, independently, and a test compares the two verdicts — a tool agreeing with itself is not evidence.

What replaced what

struct

binary record framing

zlib.crc32

record integrity

fcntl.flock

single-writer locking

os.replace

atomic replacement

json

value encoding

argparse

command-line interface

unittest

test framework

subprocess + signal

real crash testing

ast

dependency auditing

zipfile

deterministic build

STDLIB.md documents 12 genuine standard-library substitutions.

What was actually tested

The count is not the point — the hostility of the inputs is.

369

tests

passing on every supported version

12,734

hostile recovery cases

truncation, bit flips, seeded damage

12 of 12

injected defects caught

during development

Precisely: during development, 12 injected recovery defects were caught by the test suite. The mutants were scratch edits — no mutation-testing harness ships in the repository.

✓ Truncation	every byte offset of five fixtures	✓ Compaction crashes	nine named interruption points
✓ Bit corruption	every single-bit flip, expectations from an independent oracle	✓ Cross-version	CPython 3.10, 3.11, 3.12, 3.13, plus -O
✓ Seeded corruption	multi-bit, byte substitution, range damage, truncation + corruption	✓ Isolated interpreter	full suite under -E -s -S
✓ Crash recovery	real subprocesses, real SIGKILL, pipe handshake — no sleeps, no timing	✓ Standalone artifact	full workflow with the repository absent

The recovery matrices compute their expected results arithmetically from the format specification, never by calling the reader — so the tests cannot pass by agreeing with the code they test.

One file. Same bytes, every time.

```
$ python3 tools/build.py --verify
```

```
build 1: c8ae823b... (15453 bytes)
```

```
build 2: c8ae823b... (15453 bytes)
```

```
reproducible: identical bytes
```

```
dist/ledger.pyz · 15,453 bytes · SHA-256
```

```
c8ae823ba7acfe2813d9ff59f8fc0d54db0f63378e0d4332eec401e4a2cf96d7
```

The result is a small standalone artifact containing the runtime entry point and Ledger itself.

Deliberate scope

- not a SQLite replacement
- not a distributed database
- not a sync engine
- not a relational store

It replaces the fragile JSON-file, shelve and diskcache-style pattern — nothing wider.

Ledger is intentionally not a replacement for SQLite, Redis, or PostgreSQL. It addresses a narrower problem: reliable local state persistence when you want crash recovery without a third-party runtime dependency.